

Projeto Escola EAD – Programação Orientada a Objetos em Java

Desenvolver um sistema “Console” integrado para uma escola de Educação a Distância (EAD), aplicando conceitos de Programação Orientada a Objetos (POO) em Java, de forma evolutiva e prática. O projeto será construído em cinco etapas interligadas, formando um único sistema funcional ao final.

Estratégia Recomendada: Projeto Único Evolutivo

- Crie um único projeto no BlueJ chamado: ProjetoEscolaEAD
- Não crie projetos separados (Parte01, Parte02, etc.)
- A cada parte, adicione ou atualize classes existentes
- No final, tudo deve funcionar junto em um sistema com menu interativo no console
- Utilizar conceitos POO: encapsulamento, herança, sobrescrita de métodos, validações
- Trabalhos idênticos serão desconsiderados

Total: 2,5 pontos (distribuídos em cada parte)

Parte 01: Estrutura Básica (0,5 ponto)

Objetivo: Criar a classe base do sistema: Aluno.

Tarefas

Crie a classe Aluno com os atributos:

- int codigo
- String nome
- String dataNascimento
- String email
- String senha

Implemente:

- Construtor com todos os parâmetros
- Métodos get() e set() para cada atributo
- Método void exibeDados() → imprime todos os dados do aluno

Parte 02: Estruturas de Seleção e Repetição (0,5 ponto)

Objetivo: Introduzir a classe Curso e permitir cadastro múltiplo com controle de fluxo.

Tarefas

Crie a classe Curso com os atributos:

- int codigo
- String nome
- int duracao (em horas)

Métodos:

- Construtor, getters, setters, exibeDados()

Na classe SistemaEscolaEAD:

- Use um laço (for, while ou do-while) para cadastrar até N cursos
- Associe alunos aos cursos usando um dos modelos abaixo:
 - Opção A: Adicione o atributo Curso cursoMatriculado na classe Aluno
 - Opção B: Use uma matriz bidimensional Curso[][] matrizCursos (ex: por turma ou semestre) – obrigatório usar array bidimensional se escolher esta opção

- Exiba os cursos e seus alunos matriculados

Use if, switch, for, while ou do-while conforme necessário.

Exemplo de saída:

Curso: Java Básico | Duração: 40h

Alunos matriculados:

- Carlos (código 1)
- Ana (código 2)

Regra técnica:

- O uso de array bidimensional é obrigatório em pelo menos um contexto no sistema.

Parte 03: Herança, Polimorfismo e Arrays (0,5 ponto)

Objetivo: Aplicar herança, polimorfismo e gerenciar listas com arrays.

Tarefas

Crie a classe AlunoBolsista como subclasse de Aluno:

- Atributo adicional: String tipoBolsa
- Sobrescreva o construtor usando super
- Sobrescreva o método exibeDados() para incluir o tipo de bolsa

Crie a classe ListaDeAlunos:

Atributos:

- private Aluno[] alunos; // Array unidimensional obrigatório
- private int totalAlunos;

- Construtor: ListaDeAlunos(int capacidade)

- Métodos:

boolean adicionarAluno(Aluno a) → verifica duplicidade por código
void exibirLista() → percorre o vetor e chama exibeDados()

Crie a classe SistemaEscolaEAD com menu interativo (use Scanner):

=== MENU ===

- 1 - Visualizar Lista de Alunos
- 2 - Adicionar Aluno
- 3 - Sair

Observações técnicas:

- O atributo alunos[] deve ser um array unidimensional (Aluno[]), não ArrayList.
- O uso de ArrayList é permitido apenas como apoio secundário (ex: leitura temporária), mas NÃO substitui o array obrigatório nesta estrutura principal.

Parte 04: Lançamento de Notas e Cálculo de Média (0,5 ponto)

Objetivo: Gerenciar notas e médias dos alunos usando vetores (arrays unidimensionais).

Atualizações na classe Aluno

Novos atributos:

- private double[] notas = new double[3]; // Array unidimensional obrigatório
- private boolean[] lancada = new boolean[3]; // Controle de lançamento

- Declaração correta dos vetores

Métodos adicionados:

- public void lancarNotas
- public double calcularMedia() → retorna média aritmética das 3 notas
- public void exibirNotas() → mostra as notas e a média

Atualização do menu (SistemaEscolaEAD)

Nova opção:

=== MENU ===

4 - Verificar Notas do Aluno

- Solicita o código do aluno
- Busca no vetor alunos[] da ListaDeAlunos
- Mostra as 3 notas e a média calculada

Regras técnicas:

- O uso de double[3] é obrigatório. Não use ArrayList<Double>.
- Valide se o aluno existe antes de exibir notas.

Parte 05: Controle Financeiro (0,5 ponto)

Objetivo: Implementar gestão de mensalidades com arrays e controle de pagamentos.

Classe: Mensalidade

Atributos:

- double valor
- boolean pago

Métodos:

- Construtor
- Getters e setters
- void darBaixa() → marca como pago

Atualização na classe Aluno

Atributo:

- private Mensalidade[] mensalidades; // Array unidimensional obrigatório (ex: 12 parcelas)
- private int numParcelas;

Métodos:

- void adicionarMensalidades(double[] valores) → inicializa o vetor de mensalidades
- void exibirMensalidades() → mostra valor e status (pago ou não)
- void pagarMensalidade(int indice) → dá baixa na posição

Atualização do menu

Nova opção:

=== MENU ===

5 - Verificar Financeiro do Aluno

- Pede o código do aluno
- Exibe todas as mensalidades (valor, status)
- Permite pagar uma parcela específica (digitar número da parcela)

Regras técnicas:

- O atributo mensalidades[] deve ser um array unidimensional, não ArrayList.
- O uso de ArrayList<Mensalidade> é proibido como estrutura principal.
- É permitido usar ArrayList apenas como auxílio pontual (ex: conversão interna), mas NÃO substitui o array obrigatório.

Você deve entregar:

- Um projeto BlueJ único: ProjetoEscolaEAD
- Todas as 5 partes funcionando juntas em um sistema integrado
- Sistema com menu principal navegável (opções 1 a 5)
- Uso correto de:

Herança e polimorfismo	→ Sim (Aluno → AlunoBolsista)
Array unidimensional	→ Sim (notas[], alunos[], mensalidades[])
Array bidimensional	→ Sim (matriz de cursos ou similar)
Estruturas de repetição	→ Sim (for, while, do-while)
Estruturas de seleção	→ Sim (if, switch)
Encapsulamento	→ Sim (getters, setters, private)

- Trabalhos iguais serão anulados
-